PII Detection within Text Artifacts

Optiv VIT — Case Study 2 · Technical Report (feature-by-feature)

A fully-offline system that strips Personally Identifiable Information (PII) out of scanned PDFs, Word documents and PowerPoint decks **before** they are sent to an LLM document-analysis agent, and then **independently verifies that nothing leaked**. This report explains every feature of the system: what each one is for (its aim) and how it works.

Verified current state: 1028 findings · 147 unique people · 67 figures/tables sealed · **0 residual leaks** · gold recall 21/25 (84.0%) · effective coverage 25/25 (100.0%) · 39 automated self-tests green · runs fully offline.

1. The problem and the requirements

Cadence (a global financial firm) lets employees run business documents through an AI document-analysis agent, but policy forbids any personal data from reaching the model. Every document must therefore be stripped of PII first — across messy real-world formats: a 35-page scanned policy PDF, a Word training document full of screenshots, and a PowerPoint org pack. The brief's targets: near-100% PII coverage, minimal false positives, at least 80% of document structure preserved, and full traceability of every finding.

2. Architecture — a hybrid, fail-closed, 6-stage pipeline

Documents are extracted format-by-format (OCR for the scanned PDF), PII is detected with a hybrid of deterministic rules + checksums and machine-learning name recognition, every value is replaced by a stable token, regions that cannot be verified are sealed whole, and finally an independent gate re-reads and re-OCRs the output to prove no PII survives. The six stages:

- **1 · Extract** — format-aware text + layout (Poppler, Tesseract, OpenCV, python-docx/pptx)
- **2 · Detect & classify** — regex + checksums + spaCy NER + name heuristics
- **3 · Resolve / pseudonymize** — stable deterministic tokens shared across all files
- **4 · Redact (fail-closed)** — in-place token masking + whole-region sealing
- **5 · Report** — inventory, re-identification key, metrics, run metadata
- **6 · Verify (leakage gate)** — re-open text AND re-OCR the masked PDF; assert 0 PII survives

3. Stage 1 — Extraction (format-aware)

3.1 PDF rendering + OCR

Aim: *turn a scanned (image-only) PDF into machine-readable words with pixel coordinates so PII can be found and masked.*

How: Poppler renders every page to a 300-dpi PNG; Tesseract OCRs each page to a TSV of words with bounding boxes and confidence. Results are cached so re-runs take ~1 minute instead of 10–15.

3.2 OpenCV region classification

Aim: *handle PII differently depending on where it sits on the page — body text, a callout, a table, or a figure.*

How: Colour/shape heuristics in OpenCV detect gold/red callout bars (narrative), grey-bordered boxes (figures), and navy table headers extended through grey row separators (tables); every OCR word is tagged with its region type, which drives masking and sealing decisions downstream.

3.3 Enhanced second-pass OCR (figures & dense tables)

Aim: *recover text that the full-page OCR pass mangles or drops inside figures and multi-column tables.*

How: Each figure/table region is cropped, upscaled 2.5×, dark themes inverted, contrast normalised, and re-OCR'd. Two page-segmentation modes are unioned (auto-layout + uniform-block) because auto-layout drops regions next to barcodes/noise — this recovers IDs the single pass misses.

3.4 DOCX extraction

Aim: *pull PII out of every place it hides in a Word file, not just the visible body.*

How: python-docx reads paragraphs, table cells, headers/footers and document metadata (author, last-modified-by, title…); embedded PNG/JPG images (screenshots) are extracted and OCR'd.

3.5 PPTX extraction

Aim: *catch PII in the parts of a deck that are easy to forget.*

How: python-pptx reads shape text, table cells, speaker notes and metadata — notes and metadata are common leak points that manual review skips.

4. Stage 2 — Detection & classification (hybrid engine)

Deterministic rules + checksums give precision and explainability for structured identifiers; machine learning handles the fuzzy ones (names). Every finding is tagged with its category, sensitivity tier, confidence and the exact rule that caught it.

4.1 Checksum validators (Luhn, PESEL, SSN range)

Aim: *prove a candidate is a real identifier rather than a coincidental number — near-zero false positives.*

How: Credit cards must pass the Luhn algorithm; Polish PESELs must pass their weighted checksum; US SSNs must fall in a valid issued range (or appear next to an SSN keyword). A number that fails the maths is rejected, which is both precise and auditable.

4.2 Structured-identifier regex

Aim: *detect the full range of structured PII with explainable, format-specific patterns.*

How: Patterns cover email (including OCR-split *name @domain*), phone (including OCR-wrapped fragments), India PAN (structure + entity character), passport, TIN, masked/partial cards, account and SSN last-4, DOB (date + keyword), home address, IPv4, employee/vendor IDs and badge credentials — each with its own confidence and rule tag.

4.3 Machine-learning name recognition

Aim: *find people's names, which have no fixed format and defeat rules alone.*

How: spaCy NER proposes PERSON spans; these are filtered by name-pattern rules (first-initial-last, initial+surname, period-less initials, plain First-Last), a word-frequency model (tells a real surname from a common word), and a stop-word list of generic org/role terms.

4.4 Gazetteer propagation

Aim: *ensure a name confirmed once is caught everywhere it appears, including inside screenshots and prose.*

How: Every high-confidence full name seeds a gazetteer that is re-scanned across all three documents; confirmed names re-seed it for a second pass, catching repeats the first pass missed.

4.5 OCR-tolerant detection

Aim: *still catch IDs and keywords when the scanner has slightly garbled the characters.*

How: Fuzzy keyword matching (edit-distance) plus confusable-character repair (O→0, l→1, S→5…) recover SSN/PAN/passport/phone values inside noisy image text.

4.6 De-overfitting

Aim: *make detection generalise to unseen documents rather than memorising this corpus.*

How: No company, vendor, city or country names are hard-coded; organisations are rejected by generic suffix stop-words and dynamic spaCy ORG/GPE entities. Proven on an unseen synthetic document (test_generalize.py): new people caught, new companies/cities correctly ignored.

4.7 Confidence, tier and rule tagging

Aim: *give the compliance team full traceability for every finding.*

How: Each span carries a confidence score, a sensitivity tier (Critical/High/Medium) and the name of the rule that produced it — all written to the inventory. Overlapping spans are resolved by a category-priority ordering so the strongest, most specific match wins.

5. Stage 3 — Resolve & pseudonymize

5.1 Stable cross-file tokens

Aim: *preserve meaning for the LLM while removing identity — and keep the same person the same everywhere.*

How: Each value maps to a deterministic token (PERSON-016, EMAIL-002, TEL-020…) shared across all three files, so cross-document references stay linked without exposing the real value.

5.2 OCR-variant merging

Aim: *stop one person from becoming several tokens because the scanner spelled their name differently.*

How: Surname keys within edit-distance 2 are merged (e.g. Szewczyk ≈ Szewezyk, Kulkarni ≈ Kudkarni), and first-name-only mentions resolve to the right person within the same document.

5.3 Email-to-person linking

Aim: *tie a contact detail back to its owner in the re-identification key.*

How: An email's local part is matched to a known person key so the email token aligns with that person.

6. Stage 4 — Redaction (fail-closed)

6.1 In-place token masking (text)

Aim: *remove the PII while keeping the surrounding context the LLM needs to analyse the document.*

How: Detected spans in DOCX/PPTX text, tables, notes and metadata are replaced in place by their token; structural units and their order are untouched, so 80%+ of structure is preserved.

6.2 Whole-figure sealing

Aim: *guarantee no PII escapes through an image, where OCR recall is unreliable.*

How: Every embedded figure is sealed as a redaction panel — even with no detection — because OCR can render a name inside an image that it never actually recognised as text. Charts are sacrificed: the correct trade for a PII gate.

6.3 Whole-table sealing (scanned)

Aim: *stop names leaking from multi-column scanned tables that OCR renders but cannot cleanly read.*

How: Any scanned-PDF table region that contains detected PII is sealed whole, because multi-column OCR can place a value it never recognised as a maskable word (e.g. a reporter-name column).

6.4 Image-only masked PDF

Aim: *ensure there is no recoverable text hidden underneath the redaction boxes.*

How: The masked PDF is rasterised to images (via img2pdf) — the output carries no text layer at all, so nothing can be copied out from under a box.

6.5 Embedded-image & metadata redaction (Office)

Aim: *close the Office-format leak paths that aren't visible body text.*

How: Embedded screenshots in DOCX/PPTX are sealed whole inside the saved file, and author / last-modified-by metadata fields are replaced with tokens.

7. Stage 5 — Reporting & traceability

7.1 PII inventory

Aim: *give auditors a complete, traceable record of every finding.*

How: pii_inventory.csv lists every finding with file, location, context, method, category, tier, value, token, confidence and rule. (This is the companion 'PII Findings Inventory' PDF.)

7.2 Re-identification key

Aim: *allow authorised re-identification while keeping it strictly separate from the masked files.*

How: person_token_map.csv maps each token to its real name(s) and the files it appears in — kept private, never bundled with the redacted output.

7.3 Metrics & run metadata

Aim: *quantify coverage honestly and record exactly what the run did.*

How: metrics.json reports gold-set recall, effective coverage, structure retention, tier/category counts and residual leaks; run_meta.json records page metadata, figure redactions and flagged uninspected media.

7.4 Uninspected-media flagging

Aim: *surface what the system could NOT verify instead of silently passing it.*

How: Embedded vector images (EMF/SVG/WMF) that cannot be OCR'd are flagged for manual review rather than assumed clean — a PII gate must declare its blind spots.

8. Stage 6 — Verification (the independent leakage gate)

8.1 Re-read + re-OCR leakage gate

Aim: *do not trust the redactor — independently prove no detected PII value survives in the actual output.*

How: After masking, the gate re-opens the masked DOCX/PPTX text AND re-runs OCR on the rasterised masked PDF, then asserts that zero detected PII values appear in what actually renders. The first run of this gate caught leaks that manual review and two external audits had all missed (an OCR-split email and a table-rendered name). It validates masking, not detection misses.

9. Results (current, independently verified)

1028 findings · **147** unique people · **67** figures/tables sealed · **0** residual leaks. By file: Risk Policy (PDF) 852 · TPRM Training (DOCX) 52 · Org Pack (PPTX) 124. By tier: Medium 1006 · High 9 · Critical 13. Gold recall 21/25 (84.0%), effective coverage 25/25 (100.0%). Generalises to an unseen document; guarded by a 39-check self-test suite.

10. Design principles

- **Hybrid detection** — ML for fuzzy entities (names), deterministic rules+checksums for structured IDs.
- **Fail-closed** — if a region cannot be verified clean, seal it. Safety over appearance.
- **Independent verification** — re-read the output and prove it is clean; don't trust the redactor.
- **Traceability** — every finding carries its rule, confidence and tier.
- **De-overfit** — generic rules only, so it generalises beyond this corpus.

11. Honest limitations & roadmap

- Text redaction is bounded by OCR quality — which is exactly why unreadable regions are sealed and the gate re-OCRs the output.
- Recall is measured on a self-authored 25-item gold set; a blind, independently-annotated corpus is the next step.
- Six embedded vector images (EMF/SVG) cannot be OCR'd and are flagged for manual review.
- A transformer NER backend is wired in (--ner-model en_core_web_trf) for stronger name recall as a roadmap upgrade.
- A small amount of safe over-masking can occur — the correct direction for a redaction gate.